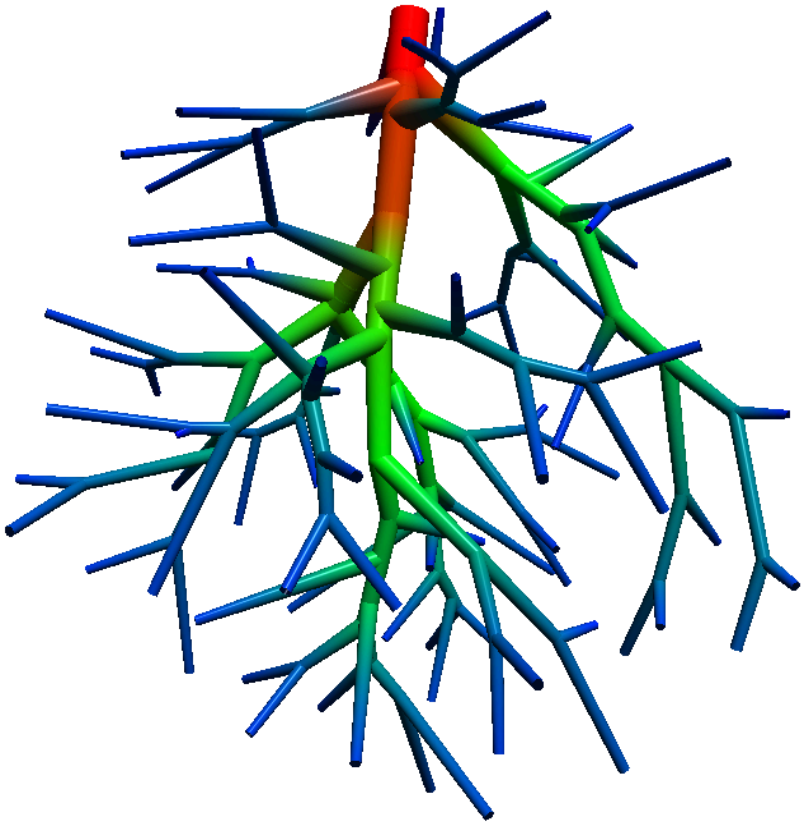
Relatório - Visualização Interativa de Árvores Arteriais (CCO)



BCC327 - Computação Gráfica

Universidade Federal de Ouro Preto (UFOP) - DECOM/ICEB

Professor: Rafael Bonfim

Aluno: Mateus Honorato de Faria - 20.2.4009

1. Introdução e Escolha Tecnológica

O objetivo central deste trabalho foi implementar uma aplicação capaz de carregar, processar e renderizar de forma interativa e visualmente realista as estruturas vasculares geradas pelo método de Otimização Coronária Construtiva (*Constructive Coronary Optimization* – CCO). Esse método produz árvores arteriais sintéticas representadas como grafos de segmentos cilíndricos, cada qual com posição, conectividade e raio próprios — dados que precisam ser lidos, transformados no espaço tridimensional e renderizados com iluminação adequada para que o resultado final seja clinicamente e visualmente significativo.

A linguagem escolhida para o desenvolvimento foi C++17, cujo padrão moderno oferece recursos como gerenciamento de memória determinístico, tipagem forte e desempenho de execução nativo, aspectos fundamentais para uma aplicação gráfica em tempo real que manipula estruturas de dados potencialmente volumosas. Como interface com o hardware gráfico, foi adotado o OpenGL 3.3 Core Profile, correspondente ao perfil de *pipeline* programável da API. Essa escolha contrasta deliberadamente com o *pipeline* fixo das versões mais antigas do OpenGL (anterior à versão 3.0), que impunha restrições severas sobre o processamento de vértices e fragmentos. No modelo programável, cada estágio de sombreamento é controlado por programas GLSL compilados em tempo de execução — denominados *shaders*. O projeto implementou dois desses programas: um *vertex shader* (vertex.glsl) e um *fragment shader* (fragment.glsl), ambos escritos em #version 330 core, que juntos suportam os três modelos clássicos de iluminação local: Phong, Gouraud e Flat Shading. A seleção entre os modelos é feita dinamicamente via *uniform* lightingMode, sem necessidade de recompilação. Para o carregamento das funções OpenGL em tempo de execução, foi utilizado o carregador GLAD, gerado para o perfil 3.3, e o gerenciamento de janela e contexto ficou a cargo da biblioteca GLFW 3.3.

Para as operações de álgebra linear — matrizes de modelo, visão e projeção, transformações de normais e cálculos de câmera —, foi utilizada a biblioteca GLM (*OpenGL Mathematics*), integrada ao projeto via CMake (find_package(glm REQUIRED)). A GLM segue as convenções de tipos e operações do GLSL, tornando trivial a transferência de dados entre a CPU e os *shaders* por meio de *uniforms*. A interface gráfica interativa foi construída sobre a biblioteca Dear ImGui, que fornece um sistema de interface de modo imediato (*immediate mode GUI*) integrado diretamente ao contexto OpenGL existente via *backends* GLFW e OpenGL3. Essa abordagem eliminou a necessidade de um *toolkit* gráfico externo pesado, mantendo toda a lógica de UI dentro do próprio *loop* de renderização.

Um fator determinante nas escolhas tecnológicas foi o ambiente de desenvolvimento utilizado: o sistema operacional Linux CachyOS (distribuição baseada em Arch Linux) com ambiente gráfico KDE Plasma executando sobre o protocolo de exibição Wayland. Diferentemente do servidor X11, o Wayland é um protocolo moderno que não expõe diretamente uma API OpenGL — dependendo de extensões como EGL e do suporte nativo das bibliotecas gráficas. Todo o *stack* escolhido (GLFW, GLAD, OpenGL 3.3 via GLVND) oferece compatibilidade nativa com Wayland por padrão, o que foi confirmado pela diretiva set(OpenGL_GL_PREFERENCE GLVND) no CMakeLists.txt, que instrui o CMake a preferir a implementação moderna e agnóstica de fornecedor das bibliotecas OpenGL. Essa escolha evitou conflitos de compatibilidade e garantiu que a aplicação funcionasse corretamente no ambiente de desenvolvimento sem recorrer a camadas de compatibilidade como XWayland.

2. Formato dos Dados e Estrutura de Leitura

Os dados de entrada do visualizador são arquivos no formato VTK Legacy (ASCII), versão 3.0, contendo a representação completa de uma árvore arterial gerada pelo método CCO. Cada arquivo segue uma estrutura sequencial bem definida: um cabeçalho de quatro linhas (identificador de versão, título, tipo ASCII e tipo de *dataset* POLYDATA), seguido de três seções de dados fundamentais — POINTS, LINES e SCALARS. A seção POINTS declara o número total de nós e, em seguida, lista as coordenadas (x,y,z) de cada vértice da árvore em ponto flutuante. A seção LINES descreve a conectividade dos segmentos arteriais: cada entrada possui o formato 2 $index_A$ $index_B$, indicando que o segmento é uma aresta entre os nós de índice $index_A$ e $index_B$. Por fim, a seção SCALARS, declarada com o nome radius (ou raio), armazena um valor escalar por segmento — o raio do cilindro arterial correspondente — lido após a linha de metadados LOOKUP_TABLE default.

A leitura desses arquivos é realizada pela classe VtkReader, implementada em VtkReader.cpp. O método load() opera como uma máquina de estados com os estados SEEK, POINTS, LINES, SCALARS e RADII. Em SEEK, o leitor avança linha a linha em busca de palavras-chave reconhecidas, comparando-as em maiúsculas para garantir robustez (*case-insensitive*). Ao identificar POINTS, transita para o estado homônimo e lê as coordenadas dos nós, populando o vetor outTree.nodes. Ao encontrar LINES, transita para o estado correspondente e constrói objetos ArterialSegment com os índices de conectividade, armazenando-os em um vetor temporário tempSegments. Somente ao localizar SCALARS radius e a subsequente LOOKUP_TABLE, o estado RADII é ativado, e os raios são atribuídos um a um aos segmentos já criados em tempSegments, via acesso direto por índice (tempSegments[radiiRead].radius = r). Ao

final, o método valida que as três seções foram efetivamente encontradas antes de confirmar os dados em outTree.segments. Esse design garante tolerância à ordem parcial das seções e rejeita arquivos incompletos com mensagens de erro descritivas.

Após o carregamento, o método VtkReader::load() invoca imediatamente outTree.normalize(), implementado em ArterialTree.cpp, cuja responsabilidade é trazer a geometria carregada para o volume canônico do OpenGL. O processo ocorre conforme estas etapas:

1. Computação da caixa delimitadora (*bounding box*) de todos os nós;
2. Determinação da maior dimensão maxDim entre os eixos *x, y* e *z*;
3. Cálculo do fator de escala scaleFactor = 2.0 / maxDim, que garante que a árvore caiba em um cubo de lado 2 centrado na origem;
4. Centralização na posição do nó raiz (nodes[0]), subtraindo sua posição antes de aplicar a escala;
5. Escala proporcional de todos os raios, com uma heurística corretiva adicional — se o maior raio escalado ultrapassar 0.2 unidades, um fator de correção é aplicado (fixFactor = 0.05 / maxRadiusScaled) para evitar que os cilindros dominassem visualmente a cena.
6. Por fim, após a normalização, os pontos médios de cada segmento são calculados e armazenados no campo seg.midpoint, utilizados posteriormente para operações de *picking* interativo.
7. O resultado é uma estrutura de dados geometricamente consistente, pronta para ser enviada à GPU.

3. Transformações Geométricas e Recorte (TP1 e TP2)

O pipeline gráfico do visualizador é inteiramente fundamentado no modelo clássico de transformações Model-View-Projection (MVP), implementado com a biblioteca GLM. A matriz Model é construída a cada quadro via glm::rotate(), aplicando uma rotação de −90° em torno do eixo *X* quando o modo 3D está ativo — correção necessária porque os dados VTK do TP2 utilizam o eixo *Z* como eixo de altura, enquanto o OpenGL usa *Y*. No modo 2D (*wireframe*), a matriz Model permanece como identidade. A matriz View é fornecida pela classe Camera através de glm::lookAt(), posicionando o observador ao redor da origem. A matriz Projection é recalculada a cada quadro com base na razão de aspecto: glm::ortho() para o modo ortográfico (TP1) e glm::perspective() com FOV de 45° para o modo perspectivo (TP2). As três matrizes são enviadas à GPU como *uniforms* (shader.setMat4("model", model), "view" e "projection"), onde o *vertex shader*

as compõe na transformação final:

$$gl_Position = P \cdot V \cdot M \cdot \vec{v}.$$

A câmera orbital é implementada na classe Camera utilizando ângulos de Euler — especificamente *Yaw* (rotação horizontal em torno do eixo *Y*) e *Pitch* (inclinação vertical). O estado da câmera é mantido por três escalares: yaw, pitch e distance. A partir deles, a posição cartesiana do observador é recalculada a cada quadro pelo método getPosition(), conforme as equações de coordenadas esféricas:

$$x = d \cdot \cos(\theta_{pitch}) \cdot \cos(\theta_{yaw}), \quad y = d \cdot \sin(\theta_{pitch}), \quad z = d \cdot \cos(\theta_{pitch}) \cdot \sin(\theta_{yaw})$$

onde *d* é a distância radial (distance). Ao arrastar o botão esquerdo do mouse, os offsets de movimento são acumulados diretamente em yaw e pitch, com o *pitch* limitado ao intervalo [−89°,+89°] para evitar o problema de inversão no polo (*gimbal lock* parcial). O *zoom* é implementado multiplicando distance por um fator de 0,9 ou 1,1 a cada evento de rolagem, e o *pan* opera no espaço da câmera: uma translação panOffset é pré-multiplicada à esquerda da matriz View (panTransform * view), garantindo que o deslocamento lateral ocorra sempre nos eixos da tela, independente da rotação atual.

O recorte de segmentos é implementado na classe ClippingUtils por meio do algoritmo de Liang-Barsky, uma técnica de recorte paramétrico especialmente eficiente para segmentos de reta contra volumes do tipo *axis-aligned bounding box* (AABB). Dado um segmento definido por:

$$P(t) = P_0 + t(P_1 - P_0), \text{ com } t \in [0, 1]$$

o algoritmo mantém um intervalo $[t_0,t_1]$ que representa a porção visível do segmento. Para cada um dos três eixos, são calculados os parâmetros *t* de interseção da reta com os dois planos delimitadores do eixo:

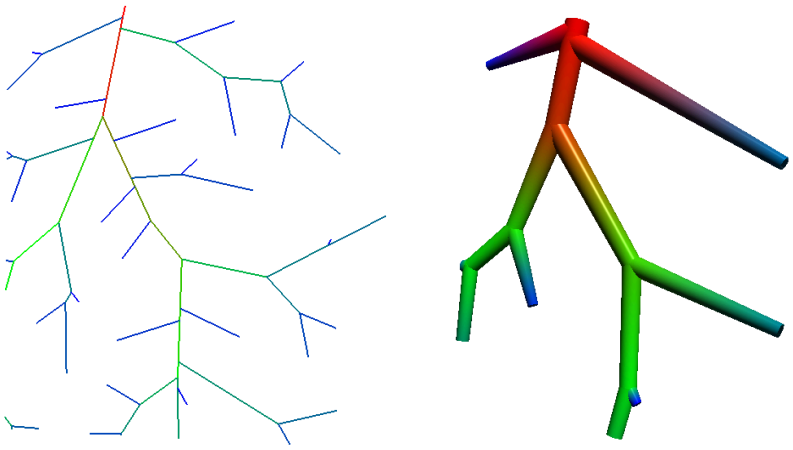
$$t_{min} = (box_{min} - P_{0,axis})/d_{axis} \text{ e } t_{max} = (box_{max} - P_{0,axis})/d_{axis}$$

Quando $d_{axis}>0$, t_{min} atualiza a entrada (t_0) e t_{max} e a a saída (t_1); caso contrário, os papéis se invertem. Se em qualquer eixo a condição $t_0>t_1$ for satisfeita, o segmento está completamente fora do volume e é descartado. Quando o segmento é paralelo a um eixo ($d_{axis}=0$), verifica-se apenas se $P_{0,axis}$ está dentro dos limites. Ao final, as

coordenadas recortadas são reconstruídas como:

$$P_0' = P_0 + t_0 \cdot \vec{p} \text{ e } P_1' = P_0 + t_1 \cdot \vec{p}.$$

O algoritmo é aplicado no espaço do objeto, antes de qualquer transformação MVP, tanto na etapa de construção das geometrias em TreeRenderer quanto na etapa de *picking* em main.cpp, garantindo consistência entre o que é desenhado e o que pode ser selecionado.



Árvores com cortes nos eixos x e y, no modo wireframe (esquerda) e cilindros (direita)

4. Renderização 3D, Iluminação e Interação (TP2)

A geometria tridimensional da árvore arterial é construída pela classe TreeRenderer, implementada em TreeRenderer.cpp. Cada segmento arterial é tesselado em um cilindro cônico com resolução de 32 faces laterais, cujos vértices são calculados por base ortonormal: a partir do vetor diretor do eixo â=normalize(B−A), dois vetores perpendiculares side e ortho são obtidos por produto vetorial, formando o plano da seção transversal. As esferas de junção nas bifurcações também são geradas por tesselação paramétrica com resolução 32×32, garantindo continuidade visual entre ramos adjacentes. Cada vértice carrega quatro atributos enviados ao VAO/VBO: posição, normal de superfície, cor (derivada de um mapa de calor que mapeia o raio do segmento numa gradação azul→verde→vermelho) e um identificador inteiro de segmento (segmentID), usado pelo *fragment shader* para destacar o segmento selecionado. O suporte à transparência é habilitado via glEnable(GL_BLEND) com

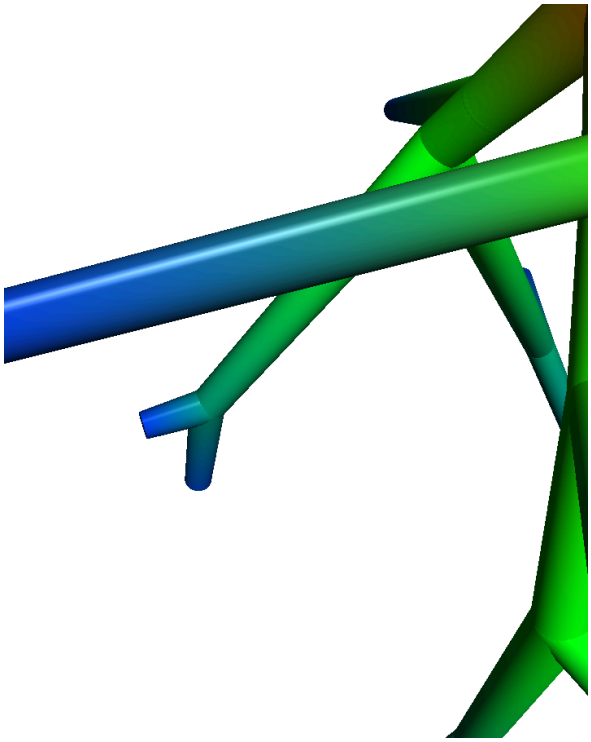
função GL_SRC_ALPHA / GL_ONE_MINUS_SRC_ALPHA, permitindo ajuste em tempo real pelo *uniform* alpha.

Os três modelos de iluminação local são implementados inteiramente em GLSL e selecionados pelo *uniform* inteiro lightingMode. No modo Phong (lightingMode == 0), todos os cálculos são realizados *per-fragment* no fragment.glsl: a cor final é a soma de três componentes — ambiente (k_a=0.2), difusa (lei de Lambert: $\max(\hat{n} \cdot \hat{l},\ 0)$ e especular (k_s=0.5, expoente de brilho α=32). A componente especular utiliza o vetor de reflexão exata $\vec{r} = \text{reflect}(-\hat{l}, \hat{n})$ avaliado com $(\hat{v} \cdot \vec{r})^{32}$, produzindo um realce de brilho coerente com a curvatura da superfície cilíndrica. No modo Gouraud (lightingMode == 1), o cálculo idêntico é realizado no vertex.glsl, armazenando a cor iluminada em GouraudColor, que é então interpolada linearmente pela rasterização até os fragmentos — resultando em brilhos suavizados mas potencialmente perdidos em polígonos grandes. No modo Flat (lightingMode == 2), a normal da face é recalculada *per-fragment* via cross(dFdx(FragPos), dFdy(FragPos)), derivadas parciais fornecidas diretamente pelo GLSL, produzindo um sombreamento facetado sem interpolação de normais entre vértices.

A remoção de superfícies ocultas é gerida pelo Z-Buffer do OpenGL, habilitado com glEnable(GL_DEPTH_TEST) durante a inicialização e limpo a cada quadro com glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT). Com o teste de profundidade ativo, o hardware gráfico descarta automaticamente qualquer fragmento cujo valor *z* normalizado seja maior (mais distante) que o valor já armazenado no *depth buffer* para aquele pixel. Isso garante que, em cenas complexas com centenas de cilindros sobrepostos — como numa árvore arterial com 512 terminais —, apenas o fragmento mais próximo do observador contribua para a cor final, sem necessidade de ordenar os polígonos por profundidade na CPU.

A seleção interativa de segmentos por clique do mouse é implementada em PickingUtils.cpp por meio da técnica de Ray Casting. Ao capturar um evento de clique, as coordenadas de tela (x_{mouse},y_{mouse}) são convertidas para o espaço do mundo utilizando glm::unProject, que aplica a inversa concatenada das matrizes de Projeção e View: o mesmo ponto de tela é desprojetado em duas profundidades distintas — z_{NDC}=0 (plano próximo) e z_{NDC}=1 (plano distante) — gerando dois pontos no espaço tridimensional, nearPoint e farPoint. O raio de seleção é então definido pela origem outOrigin = nearPoint e a direção normalizada $\hat{d} = \text{normalize}(\text{farPoint} - \text{nearPoint})$. Para cada segmento visível (respeitando o estado de recorte ativo), rayIntersectsSegment()

resolve o sistema de mínima distância entre duas retas — o raio e o eixo do cilindro — parametrizados por s_c e t_c respectivamente. Se a distância mínima entre os pontos correspondentes for inferior ao raio do cilindro, há interseção, e o segmento com menor s_c (mais próximo da câmera) é marcado como selecionado, sendo destacado pelo *fragment shader* com uma mistura de amarelo (mix(result, vec3(1,1,0), 0.6)).

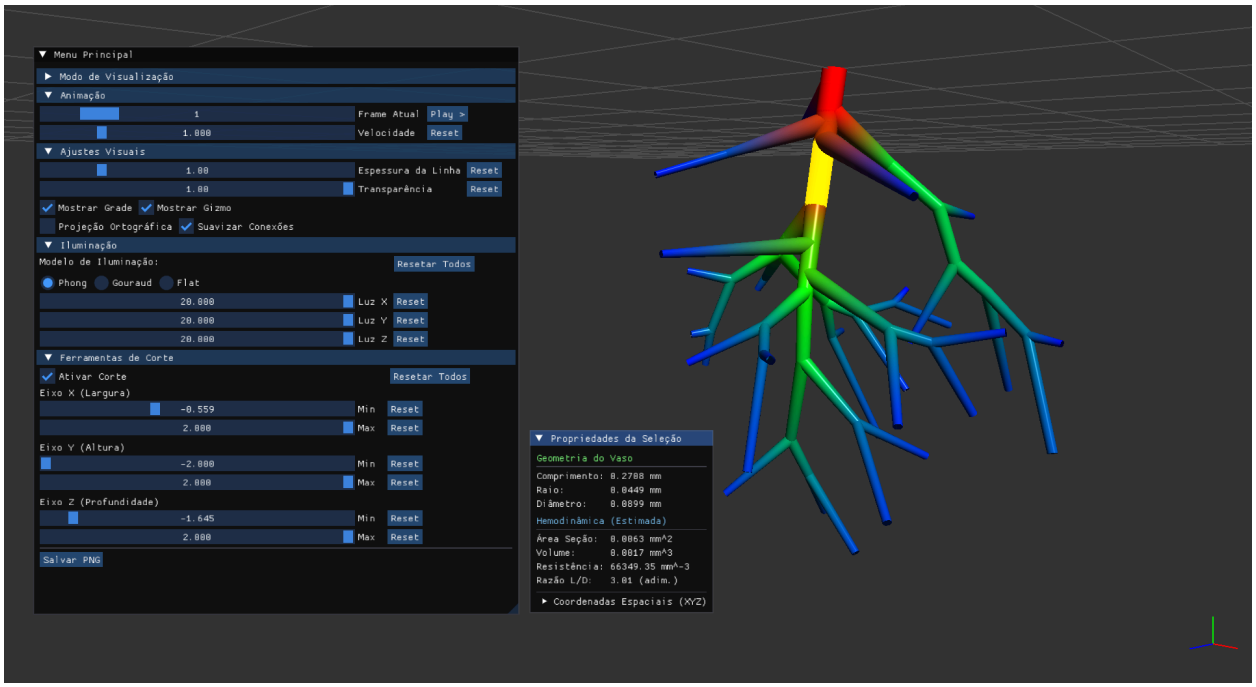


Brilho especular do Modelo de Phong

5. Resultados Visuais e Conclusão

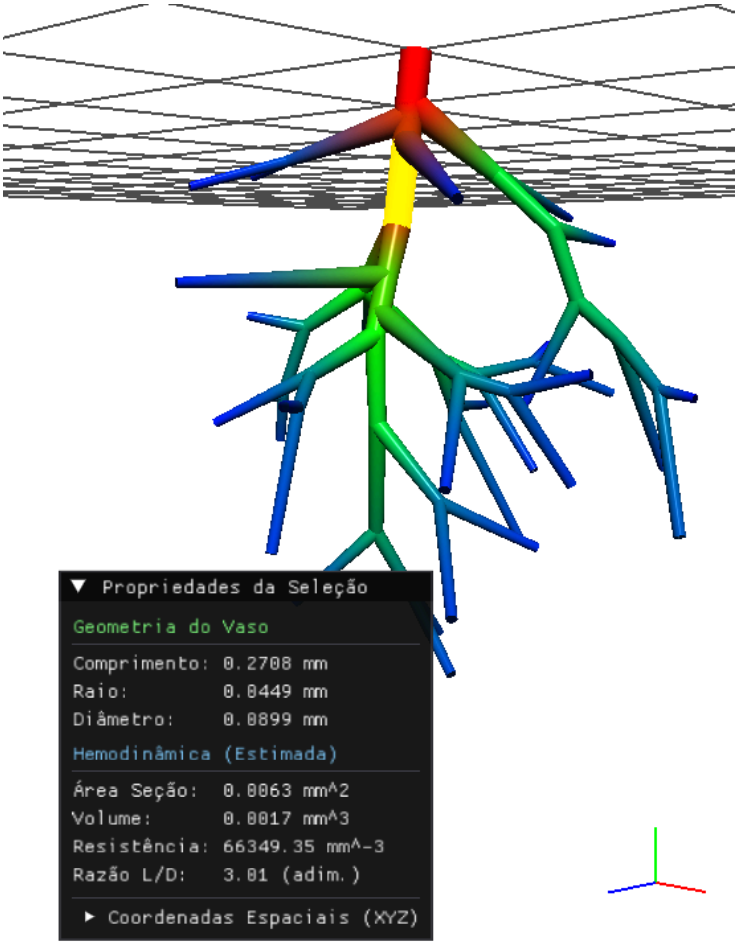
O desenvolvimento do Visualizador Interativo e Realista de Árvores Arteriais resultou em uma aplicação funcional e visualmente coerente que atende integralmente aos requisitos propostos para ambas as etapas do trabalho prático. Para o TP1, foram implementados o carregamento de arquivos VTK (2D e 3D) com validação robusta por máquina de estados, o modo de visualização *wireframe* com mapa de calor baseado no raio dos segmentos, e o recorte paramétrico por Liang-Barsky ajustável em tempo real pelos três eixos independentes via Dear ImGui. Para o TP2, o visualizador evoluiu para uma renderização 3D completa com malha cilíndrica tesselada, esferas de junção nas bifurcações, e suporte aos três modelos clássicos de iluminação local (Phong, Gouraud e Flat Shading) selecionáveis interativamente, acompanhado de controles de posição da fonte de luz, escala de raio, transparência e projeções ortográfica e perspectiva.

O painel de controle — construído com Dear ImGui e organizado em seções colapsáveis (Modo de Visualização, Animação, Ajustes Visuais, Iluminação, Ferramentas de Corte) — permanece integrado ao *loop* de renderização sem framerate perceptível.



Painel de controle e interface completa com Dear ImGui

Uma das funcionalidades de maior impacto visual é a animação temporal do crescimento CCO: o AnimationController carrega automaticamente a sequência de arquivos VTK ordenados de uma pasta (*playlist*), avançando quadro a quadro com controle de velocidade ajustável por *slider* e botão Play/Pause, reproduzindo o processo construtivo da árvore desde os primeiros segmentos até a topologia completa. A seleção interativa por *ray casting* completa a experiência analítica: ao clicar em qualquer segmento visível, uma janela flutuante exibe em tempo real as propriedades hemodinâmicas estimadas do vaso selecionado — comprimento, raio, diâmetro, área da seção transversal, volume, resistência geométrica proporcional (L/r^4) e razão de aspecto *L*/*D*, com unidades explícitas em milímetros. A seleção persiste entre quadros da animação por meio de uma heurística de rastreamento por ponto médio (lastSelectedMidpoint), garantindo que o segmento correspondente ao selecionado no quadro anterior seja reidentificado no quadro seguinte mesmo que os índices se alterem.



Seleção iterativa por ray casting

A construção deste projeto consolidou de forma prática e profunda a compreensão do *pipeline* gráfico programável. Implementar cada etapa manualmente — da leitura de dados brutos em disco até a composição final das matrizes MVP e a execução dos *shaders* na GPU — tornou concretos conceitos que, em abstrações de alto nível, permaneceriam opacos: a separação de responsabilidades entre CPU e GPU, o fluxo de atributos por VAOs e VBOs, a interpolação de normais pelo rasterizador e seu impacto direto na qualidade visual dos modelos de iluminação, e a inversão das transformações de projeção para implementar interações como o *ray picking*. O projeto demonstrou, ainda, que um visualizador científico funcional pode ser construído com um *stack* enxuto e moderno — C++17, OpenGL 3.3 Core, GLM e Dear ImGui —, sem dependências pesadas, mantendo desempenho em tempo real mesmo com árvores de 512 terminais e geometria de alta resolução.